

重点回答

1. SGLang 的 RadixAttention / RadixCache 解决什么问题？

大模型在处理用户的请求时，会把已经计算过的 K (key)、V (value) 保存在 KV Cache 中。实际业务里，不同请求经常共享相同前缀，比如相同的 system prompt、多轮对话历史、RAG 中重复的文档片段。如果每次都从头做 Prefill，就会反复计算同一段前缀，延长处理时间，产生额外的算力消耗。RadixAttention 的核心就是把这些可复用的 KV Cache 按前缀组织到一个 Radix Tree 中。新请求到来时先寻找并匹配最长可复用前缀，命中的部分直接使用已有 KV，只对后面的新 Token 做计算，也就是常说的缓存命中。这样可以减少重复的 Prefill，也能降低首 Token 等待时间，并提高服务的吞吐量。

2. page-sized KV cache 与 prefix reuse 分别对应 pipeline 中的哪一部分？

page-sized KV cache 主要属于 KV Cache 的底层内存管理。Prefill 产生 KV，Decode 解码又会不断读取并增加新的 KV；把缓存按页或块管理，可以让逻辑上连续的一段 KV 不必占用物理上连续的一大块显存，便于动态分配和回收。prefix reuse 则发生在新请求准备进入 Prefill 的时候：系统先在缓存中查找已经计算过的前缀，命中的 K、V 直接复用，因此 Prefill 只需要计算未命中的后缀。简单说，page-sized KV cache 回答的是 KV 怎么存，prefix reuse 回答的是存下来的 KV 如何使用。

3. vLLM 的 PagedAttention 解决什么问题？它和 SGLang 的 RadixAttention / paged KV 机制有什么联系与差异？

PagedAttention 主要解决 KV Cache 很大、长度动态变化时的显存浪费和碎片问题。它通过借鉴操作系统对虚拟内存进行分页存储的思路，把每个请求的 KV Cache 切成固定大小的 block，通过映射关系把逻辑连续的 KV 放到不连续的物理显存中，并按需分配，从而提高存利用率。和 SGLang 的 paged KV 在分页管理 KV cache 这一点上的思路相近。区别在于关注的重点不同：用图书馆来进行一个简单的比喻，PagedAttention 用来解决书怎么摆，强调的是空间的管理；而 RadixAttention 用来解决“别人已经有这本书的前五章了，我还需要不需要重新打印这五章？”，强调的是计算的复用。